

雷达信号处理中的矩阵方法

目 录

4	第 1 章 绪论	1
5	1.1 雷达的基本概念	1
6	1.2 雷达工作原理	2
7	1.2.1 基本组成	2
8	1.2.2 工作频率	2
9	1.2.3 目标探测	4
10	1.3 电磁波传播特性	4
11	1.3.1 电磁波的反射	5
12	1.3.2 电磁波的折射	6
13	1.3.3 自由空间路径损耗与大气衰减	8
14	1.3.4 色散现象与多径效应	9
15	1.4 雷达方程	10
16	1.5 小结	13
17	第 2 章 矩阵方法基础	15
18	2.1 矩阵分解	15
19	2.1.1 特征分解	15
20	2.1.2 奇异值分解	19
21	2.1.3 稀疏分解	22
22	2.2 矩阵微分	24
23	2.2.1 实矩阵微分	24
24	2.2.2 复矩阵微分	28
25	2.3 张量及相关运算	30
26	2.3.1 张量的定义	30
27	2.3.2 张量的运算	32
28	2.3.3 张量分解	34
29	2.4 常见统计量的矩阵表示	35
30	2.5 多元函数的泰勒展开	39

31	第 3 章 雷达发射与接收信号模型	41
32	3.1 发射信号模型	41
33	3.2 阵列接收信号模型	44
34	3.2.1 一维均匀线性阵列	44
35	3.2.2 二维均匀线性阵列	48
36	3.3 多普勒接收信号模型	50
37	3.4 空时联合接收信号模型	52
38	3.5 噪声、杂波和多径模型	54
39	第 4 章 雷达信号检测	57
40	4.1 脉冲压缩	57
41	4.2 脉冲积累	63
42	4.3 目标检测	65
43	4.3.1 门限选择与检测性能	65
44	4.3.2 恒虚警率检测	69
45	第 5 章 目标参数估计	73
46	5.1 波束形成方法	73
47	5.1.1 经典波束形成	74
48	5.1.2 和差比幅测角	78
49	5.1.3 最小方差无失真响应	81
50	5.2 子空间方法	83
51	5.2.1 MUSIC 方法	84
52	5.2.2 ESPRIT 方法	89
53	5.3 稀疏表示方法	93
54	5.4 多域联合估计	96
55	5.4.1 空时自适应处理	96
56	5.4.2 张量分解方法	101
57	第 6 章 目标跟踪	105
58	6.1 状态估计	105
59	6.1.1 最小均方滤波	105
60	6.1.2 递推最小均方滤波	107
61	6.1.3 卡尔曼滤波	110

62	6.2 数据关联	114
63	6.2.1 联合概率数据关联	115
64	6.2.2 图匹配方法	116
65	第 7 章 无源定位	121
66	7.1 测向定位	121
67	7.1.1 二维平面	121
68	7.1.2 三维空间	125
69	7.2 时差定位	128
70	7.3 频差定位	131
71	参考文献	133
72	附录 A 基础知识	135
73	A.1 符号规定	135
74	A.2 矩阵函数	135
75	A.3 卷积与相关的矩阵表示	135
76	A.4 拉格朗日乘数法	138
77	A.5 分块矩阵求逆公式	139
78	附录 B 向量范数与矩阵范数	143
79	B.1 向量范数	143
80	B.2 矩阵范数	146
81	附录 C 张量乘法性质	149

第 6 章 目标跟踪

目标跟踪 (Target Tracking) 是雷达信号处理与信息融合中的核心任务。它旨在通过连续时间序列的量测数据, 对运动目标状态进行实时估计与更新, 并在多目标和杂波干扰下维持可靠航迹。通常而言, 目标跟踪面临两个基本问题: 其一是状态估计, 即在噪声环境下对目标的动力学状态 (位置、速度、加速度等隐含量) 进行递推估计; 其二是数据关联, 即在多目标与虚警并存的量测环境中, 判断每个观测量应当归属于哪一条目标航迹。本章将围绕这两个关键问题展开, 首先介绍状态估计方法, 然后讨论数据关联方法, 从而构建目标跟踪的基本理论框架。

6.1 状态估计

在前述目标检测与参数估计的基础上, 可以获得目标的多种观测信息, 如距离、方位与速度等。然而在实际应用中, 这些观测量不可避免地受到噪声与干扰的影响, 因而存在偏差与不确定性。为了提升信息的可靠性, 需要综合利用历史测量进行处理, 实现对观测序列的降噪与平滑, 并进一步完成对目标未来状态的预测。这便构成了状态估计问题的核心任务, 也是实现高精度目标跟踪的前提。

6.1.1 最小均方滤波

不妨假设在第 t 时刻, 目标的状态向量为 $\mathbf{x}_t \in \mathbb{R}^M$, 其可包含目标的速度、加速度等多种状态变量。同时, 假设在该时刻可以获得一个观测向量 $\mathbf{z}_t \in \mathbb{R}^N$, 其可能是目标的距离、方位等测量值。在线性假设下, 观测向量与目标状态向量之间的关系通常可以使用如下模型来描述:

$$\mathbf{z}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t,$$

其中, $\mathbf{H} \in \mathbb{R}^{N \times M}$ 为待估计的观测矩阵, 用于刻画状态向量与观测向量之间的线性映射关系; $\mathbf{v}_t$ 为观测噪声, 通常假设其服从零均值高斯分布。最小二乘算法 (Least Squares, LS) 的目标是在对观测数据进行处理时, 最小化估计误差的均方值, 从而获得对目标状态的最优估计。若共获得 T 次观测, 则相应的优化问题可表述为:

$$\min_{\mathbf{H}_T} \frac{1}{T} \sum_{t=1}^T \|\mathbf{H}_T \mathbf{x}_t - \mathbf{z}_t\|^2,$$

其中, $\mathbf{H}_T$ 表示利用前 T 次观测数据估计得到的观测矩阵。

为便于表达, 定义数据矩阵

$$\mathbf{X}_T = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_T \end{bmatrix} \in \mathbb{R}^{M \times T}, \quad \mathbf{Z}_T = \begin{bmatrix} \mathbf{z}_1 & \mathbf{z}_2 & \cdots & \mathbf{z}_T \end{bmatrix} \in \mathbb{R}^{N \times T}.$$

则上述优化问题可进一步简化为 (其中常数因子 $\frac{1}{T}$ 已省略)

$$\min_{\mathbf{H}_T} \|\mathbf{H}_T \mathbf{X}_T - \mathbf{Z}_T\|_{\text{F}}^2.$$

利用克罗内克积的相关性质 (推论 C.2), $\mathbf{H}_T \mathbf{X}_T$ 可以改写为如下的矩阵乘向量的形式:

$$\text{vec}(\mathbf{H}_T \mathbf{X}_T) = \text{vec}(\mathbf{I} \mathbf{H}_T \mathbf{X}_T) = (\mathbf{X}_T^{\text{T}} \otimes \mathbf{I}) \text{vec}(\mathbf{H}_T),$$

其中, $\mathbf{I} \in \mathbb{R}^{N \times N}$ 为单位矩阵。于是, 优化问题可转化为

$$\min_{\text{vec}(\mathbf{H}_T)} \left\| (\mathbf{X}_T^{\text{T}} \otimes \mathbf{I}) \text{vec}(\mathbf{H}_T) - \text{vec}(\mathbf{Z}_T) \right\|^2.$$

该问题为标准的线性最小二乘问题, 其解析解为

$$\begin{aligned} \text{vec}(\mathbf{H}_T) &= ((\mathbf{X}_T^{\text{T}} \otimes \mathbf{I})^{\text{T}} (\mathbf{X}_T^{\text{T}} \otimes \mathbf{I}))^{-1} (\mathbf{X}_T^{\text{T}} \otimes \mathbf{I})^{\text{T}} \text{vec}(\mathbf{Z}_T) \\ &= ((\mathbf{X}_T \mathbf{X}_T^{\text{T}}) \otimes \mathbf{I})^{-1} (\mathbf{X}_T \otimes \mathbf{I}) \text{vec}(\mathbf{Z}_T) \\ &= ((\mathbf{X}_T \mathbf{X}_T^{\text{T}})^{-1} \otimes \mathbf{I}) (\mathbf{X}_T \otimes \mathbf{I}) \text{vec}(\mathbf{Z}_T) \\ &= ((\mathbf{X}_T \mathbf{X}_T^{\text{T}})^{-1} \mathbf{X}_T \otimes \mathbf{I}) \text{vec}(\mathbf{Z}_T), \end{aligned}$$

再次利用推论 C.2, 可得

$$\mathbf{H}_T = \mathbf{Z}_T \mathbf{X}_T^{\text{T}} (\mathbf{X}_T \mathbf{X}_T^{\text{T}})^{-1}.$$

注意到, 随着观测次数 T 的增加, 数据矩阵 $\mathbf{X}_T$ 的规模不断增大, 计算复杂度迅速提升。另一方面, 目标的运动状态也可能随时间发生变化, 因此没有必要始终依赖全部历史观测数据来估计观测矩阵 $\mathbf{H}_T$ 。基于此, 可以考虑采用迭代方式对观测矩阵进行逐步更新。

假设在时刻 T 已经得到了观测矩阵 $\mathbf{H}_T$, 当引入第 $T+1$ 次观测后, 仅利用该次观测来修正已有估计。相应的优化问题为

$$\min_{\mathbf{H}_T} \|\mathbf{H}_T \mathbf{x}_{T+1} - \mathbf{z}_{T+1}\|^2.$$

该目标函数关于 $\mathbf{H}_T$ 的梯度为

$$\nabla_{\mathbf{H}_T} = 2(\mathbf{H}_T \mathbf{x}_{T+1} - \mathbf{z}_{T+1}) \mathbf{x}_{T+1}^{\text{T}}.$$

其中, $\mathbf{H}_T \mathbf{x}_{T+1} - \mathbf{z}_{T+1}$ 为利用当前观测矩阵对最新观测进行预测时所产生的误差。记 $\mathbf{e}_{T+1} = \mathbf{z}_{T+1} - \mathbf{H}_T \mathbf{x}_{T+1}$, 则可采用梯度下降法更新观测矩阵。对应的更新公式为

$$\mathbf{H}_{T+1} = \mathbf{H}_T + \mu \mathbf{e}_{T+1} \mathbf{x}_{T+1}^{\text{T}},$$

其中 μ 为步长参数 ($\mu > 0$)。这种基于随机梯度下降 (Stochastic Gradient Descent, SGD) 更新的方法被称为最小均方 (Least Mean Square, LMS) 算法。可以发现其核心思想在于：每次引入新观测时，先利用当前观测矩阵进行预测并计算预测误差，随后结合该误差与当前观测向量对观测矩阵进行修正，从而实现观测矩阵动态更新。

6.1.2 递推最小均方滤波

在实际应用中，步长参数 μ 的选取对算法的收敛性与稳定性具有重要影响。若 μ 过大，可能导致算法发散；若 μ 过小，则收敛速度较慢。而递推最小均方 (Recursive Least Mean Square, RLMS) 算法通过两种方式提升性能：其一，引入遗忘因子 $\lambda \in (0, 1]$ 以减小对早期观测数据的依赖；其二，利用伍德伯里矩阵恒等式 (Woodbury Matrix Identity, 见推论 A.1) 高效地更新观测矩阵，避免步长选取。从而，算法鲁棒性得到增强。

具体而言，在时刻 T 时，优化问题可表示为

$$\min_{\mathbf{H}_T} \sum_{t=1}^T \lambda^{T-t} \|\mathbf{H}_T \mathbf{x}_t - \mathbf{z}_t\|^2.$$

同样可以将上述问题转化为矩阵形式：

$$\min_{\mathbf{H}_T} \|\mathbf{H}_T \mathbf{X}_T - \mathbf{Z}_T\|_F^2,$$

其中

$$\begin{aligned} \mathbf{X}_T &= \begin{bmatrix} \sqrt{\lambda}^{T-1} \mathbf{x}_1 & \sqrt{\lambda}^{T-2} \mathbf{x}_2 & \cdots & \mathbf{x}_T \end{bmatrix} \in \mathbb{R}^{M \times T}, \\ \mathbf{Z}_T &= \begin{bmatrix} \sqrt{\lambda}^{T-1} \mathbf{z}_1 & \sqrt{\lambda}^{T-2} \mathbf{z}_2 & \cdots & \mathbf{z}_T \end{bmatrix} \in \mathbb{R}^{N \times T}. \end{aligned}$$

与普通最小二乘相同， T 时刻该问题的解析解为

$$\mathbf{H}_T = \mathbf{Z}_T \mathbf{X}_T^T (\mathbf{X}_T \mathbf{X}_T^T)^{-1},$$

而 $T+1$ 时刻的解为

$$\mathbf{H}_{T+1} = \mathbf{Z}_{T+1} \mathbf{X}_{T+1}^T (\mathbf{X}_{T+1} \mathbf{X}_{T+1}^T)^{-1}.$$

注意到，矩阵 $\mathbf{X}_T$ 和 $\mathbf{X}_{T+1}$ ，矩阵 $\mathbf{Z}_T$ 和 $\mathbf{Z}_{T+1}$ 分别存在如下关系：

$$\mathbf{X}_{T+1} = \begin{bmatrix} \sqrt{\lambda} \mathbf{X}_T & \mathbf{x}_{T+1} \end{bmatrix}, \quad \mathbf{Z}_{T+1} = \begin{bmatrix} \sqrt{\lambda} \mathbf{Z}_T & \mathbf{z}_{T+1} \end{bmatrix}.$$

因此， $\mathbf{Z}_{T+1} \mathbf{X}_{T+1}^T$ 可以写成如下的形式：

$$\mathbf{Z}_{T+1} \mathbf{X}_{T+1}^T = \begin{bmatrix} \sqrt{\lambda} \mathbf{Z}_T & \mathbf{z}_{T+1} \end{bmatrix} \begin{bmatrix} \sqrt{\lambda} \mathbf{X}_T^T \\ \mathbf{x}_{T+1}^T \end{bmatrix} = \lambda \mathbf{Z}_T \mathbf{X}_T^T + \mathbf{z}_{T+1} \mathbf{x}_{T+1}^T.$$

类似地, $\mathbf{X}_{T+1}\mathbf{X}_{T+1}^T$ 可以写成如下的形式:

$$\mathbf{X}_{T+1}\mathbf{X}_{T+1}^T = \begin{bmatrix} \sqrt{\lambda}\mathbf{X}_T & \mathbf{x}_{T+1} \end{bmatrix} \begin{bmatrix} \sqrt{\lambda}\mathbf{X}_T^T \\ \mathbf{x}_{T+1}^T \end{bmatrix} = \lambda\mathbf{X}_T\mathbf{X}_T^T + \mathbf{x}_{T+1}\mathbf{x}_{T+1}^T.$$

方便起见, 记 $\mathbf{X}_T\mathbf{X}_T^T = \boldsymbol{\Sigma}_T$, $\mathbf{Z}_T\mathbf{X}_T^T = \boldsymbol{\Gamma}_T$, 则进一步利用伍德伯里矩阵恒等式 (推论 A.1), 可得

$$\begin{aligned} (\boldsymbol{\Sigma}_{T+1})^{-1} &= (\lambda\boldsymbol{\Sigma}_T + \mathbf{x}_{T+1}\mathbf{x}_{T+1}^T)^{-1} = \frac{1}{\lambda} \left(\boldsymbol{\Sigma}_T^{-1} - \frac{\boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}\mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1}}{\lambda + \mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}} \right) \\ &= \frac{1}{\lambda} (\boldsymbol{\Sigma}_T^{-1} - \boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}\mathbf{g}_{T+1}^T), \end{aligned}$$

其中 $\mathbf{g}_{T+1}$ 被称作增益向量, 有如下表达式:

$$\mathbf{g}_{T+1} = \frac{\boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}}{\lambda + \mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}}.$$

注意到, $\mathbf{H}_{T+1}$ 可以表示为

$$\begin{aligned} \mathbf{H}_{T+1} &= \boldsymbol{\Gamma}_{T+1}\boldsymbol{\Sigma}_{T+1}^{-1} = (\lambda\boldsymbol{\Gamma}_T + \mathbf{z}_{T+1}\mathbf{x}_{T+1}^T)\boldsymbol{\Sigma}_{T+1}^{-1} \\ &= \lambda\boldsymbol{\Gamma}_T\boldsymbol{\Sigma}_{T+1}^{-1} + \mathbf{z}_{T+1}\mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_{T+1}^{-1} \end{aligned}$$

其中, 第一项可进一步化简为

$$\lambda\boldsymbol{\Gamma}_T\boldsymbol{\Sigma}_{T+1}^{-1} = \boldsymbol{\Gamma}_T(\boldsymbol{\Sigma}_T^{-1} - \boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}\mathbf{g}_{T+1}^T) = \mathbf{H}_T - \mathbf{H}_T\mathbf{x}_{T+1}\mathbf{g}_{T+1}^T,$$

而第二项则可化简为

$$\begin{aligned} \mathbf{z}_{T+1}\mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_{T+1}^{-1} &= \frac{\mathbf{z}_{T+1}}{\lambda} \left(\mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1} - \frac{\mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}\mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1}}{\lambda + \mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}} \right) \\ &= \frac{\mathbf{z}_{T+1}\mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1}}{\lambda + \mathbf{x}_{T+1}^T\boldsymbol{\Sigma}_T^{-1}\mathbf{x}_{T+1}} \\ &= \mathbf{z}_{T+1}\mathbf{g}_{T+1}^T. \end{aligned}$$

因此, $\mathbf{H}_{T+1}$ 可表示为

$$\begin{aligned} \mathbf{H}_{T+1} &= \mathbf{H}_T - \mathbf{H}_T\mathbf{x}_{T+1}\mathbf{g}_{T+1}^T + \mathbf{z}_{T+1}\mathbf{g}_{T+1}^T = \mathbf{H}_T + (\mathbf{z}_{T+1} - \mathbf{H}_T\mathbf{x}_{T+1})\mathbf{g}_{T+1}^T \\ &= \mathbf{H}_T + \mathbf{e}_{T+1}\mathbf{g}_{T+1}^T. \end{aligned}$$

综上，递推最小均方（RLMS）算法的更新公式为

$$\left\{ \begin{array}{ll} \text{更新增益向量:} & \mathbf{g}_{T+1} = \frac{\boldsymbol{\Sigma}_T^{-1} \mathbf{x}_{T+1}}{\lambda + \mathbf{x}_{T+1}^T \boldsymbol{\Sigma}_T^{-1} \mathbf{x}_{T+1}}, \\ \text{更新协方差矩阵:} & \boldsymbol{\Sigma}_{T+1}^{-1} = \frac{1}{\lambda} (\boldsymbol{\Sigma}_T^{-1} - \boldsymbol{\Sigma}_T^{-1} \mathbf{x}_{T+1} \mathbf{x}_{T+1}^T \boldsymbol{\Sigma}_T^{-1}), \\ \text{计算预测误差:} & \mathbf{e}_{T+1} = \mathbf{z}_{T+1} - \mathbf{H}_T \mathbf{x}_{T+1}, \\ \text{更新观测矩阵:} & \mathbf{H}_{T+1} = \mathbf{H}_T + \mathbf{e}_{T+1} \mathbf{g}_{T+1}^T, \\ \text{进行预测:} & \hat{\mathbf{z}}_{T+1} = \mathbf{H}_{T+1} \mathbf{x}_{T+1}. \end{array} \right.$$

由此可见，整个递推过程同样可以理解为：在每次引入新观测时，先利用当前观测矩阵进行预测并计算预测误差，随后结合该误差与增益向量对观测矩阵进行修正，从而实现动态更新。当 $\lambda = 1$ 时，该递推公式退化为经典的最小二乘算法，再结合最小均方滤波算法的迭代公式，可见三者本质上均遵循“先预测、后修正”的思想，区别仅在于增益向量的取值不同。

例 6.1 利用最小二乘（LS）、最小均方（LMS）与递推最小均方（RLMS）算法对目标轨迹进行预测，目标轨迹如图 6.1 所示。

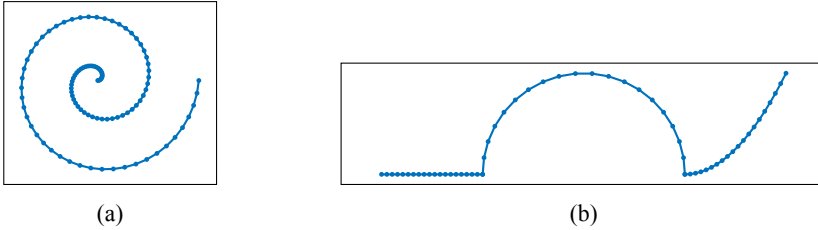


图 6.1 目标轨迹示意图

解 设 t 时刻目标的坐标为 $\mathbf{p}_t$ ，则可以将历史坐标作为状态向量，而将当前坐标作为观测向量，即

$$\mathbf{x}_t = \begin{bmatrix} \mathbf{p}_{t-1} \\ \vdots \\ \mathbf{p}_{t-k} \end{bmatrix} \in \mathbb{R}^{2k}, \quad \mathbf{z}_t = \mathbf{p}_t \in \mathbb{R}^2,$$

其中， k 为使用的历史坐标数。利用前 T 次观测数据，可以估计出观测矩阵 $\mathbf{H}_T$ ，进而对第 $T+1$ 时刻的目标位置进行预测。选取 $k=4$ 时，预测结果如图 6.2 和 6.3 所示。

可以看到，对于螺旋曲线这样的简单可预测的轨迹，直接利用 LS 算法即可获得良好的预测效果。利用 LMS 算法则可以大大降低计算复杂度，但从图 6.2b 中可以发现，其收敛速度较慢。进一步利用 RLMS 算法，则可以在较快收敛的同时，获得与 LS 算法相当的预测效果。

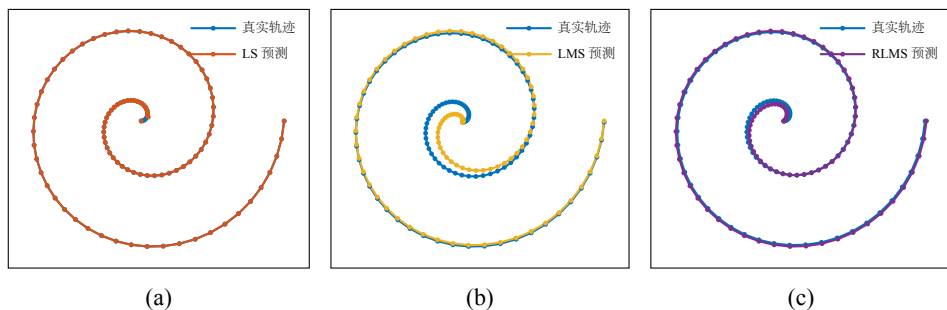


图 6.2 轨迹一预测结果

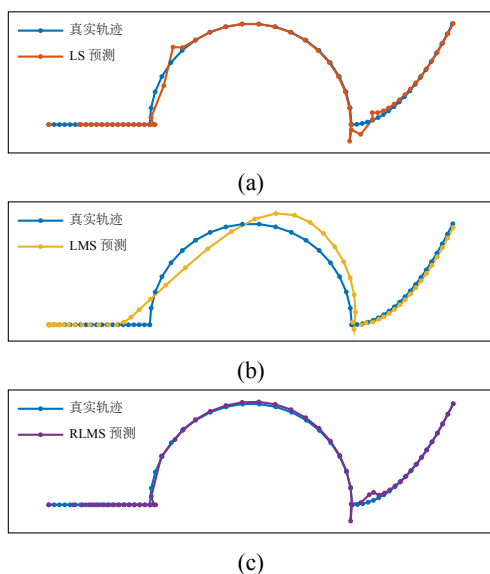


图 6.3 轨迹二预测结果

对于复杂且难以预测的轨迹，如图 6.1b 所示，直接利用 LS 算法进行预测效果较差。从图 6.3a 中可以看到，当轨迹出现突变时，LS 算法无法及时跟踪。LMS 算法收敛速度较慢，因此其预测效果（图 6.3b）也不理想。而 RLMS 算法则可以较快地适应轨迹的变化，从而获得较好的预测效果（图 6.3c）。

6.1.3 卡尔曼滤波

在实际场景中，目标的运动状态受制于物理规律，因此其状态转移通常具有一定的连续性与相关性。卡尔曼滤波（Kalman Filter, KF）算法正是利用这一点，通过引入状态转移模型来对目标状态进行预测与更新，从而提升估计的准确性与鲁棒性。

在卡尔曼滤波算法中，同样假设观测向量可以由目标状态向量线性映射得到：

$$\mathbf{z}_t = \mathbf{H}_t \mathbf{x}_t + \mathbf{v}_t,$$

其中， $\mathbf{H}_t \in \mathbb{R}^{N \times M}$ 为观测矩阵， $\mathbf{v}_t$ 为观测噪声，通常假设其服从零均值高斯分布。但与最小均方滤波不同，卡尔曼滤波还引入了状态转移模型：

$$\mathbf{x}_t = \mathbf{F}_t \mathbf{x}_{t-1} + \mathbf{B}_t \mathbf{u}_t + \mathbf{w}_t,$$

其中， $\mathbf{F}_t \in \mathbb{R}^{M \times M}$ 为状态转移矩阵， $\mathbf{B}_t \in \mathbb{R}^{M \times K}$ 为控制矩阵， $\mathbf{u}_t \in \mathbb{R}^{K \times 1}$ 为控制输入， $\mathbf{w}_t$ 为过程噪声。简而言之，卡尔曼滤波认为：目标状态取决于前一时刻状态与控制输入，并受过程噪声干扰；而观测向量由当前状态线性映射得到，亦受观测噪声干扰。

例如，对于一个匀加速直线运动的目标，令 p_t 为目标在时刻 t 的位置， v_t 为目标在时刻 t 的速度，且加速度为常数 a 。则在时刻 t 时，目标的位置可以近似表示为

$$p_t = p_{t-1} + v_{t-1} \Delta t + \frac{1}{2} a (\Delta t)^2,$$

其中， Δt 为相邻两次观测的时间间隔。与此同时，目标的速度可以近似表示为

$$v_t = v_{t-1} + a \Delta t,$$

其中 a 为目标的加速度。若将位置与速度作为状态向量，则状态转移方程可表示为

$$\begin{bmatrix} p_t \\ v_t \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_{t-1} \\ v_{t-1} \end{bmatrix} + \begin{bmatrix} \frac{1}{2} (\Delta t)^2 \\ \Delta t \end{bmatrix} a,$$

即状态转移矩阵、控制矩阵和控制输入分别为

$$\mathbf{F}_t = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad \mathbf{B}_t = \begin{bmatrix} \frac{1}{2} (\Delta t)^2 \\ \Delta t \end{bmatrix}, \quad \mathbf{u}_t = a.$$

若令观测向量即为目标的位置，则观测方程可表示为

$$\mathbf{z}_t = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} p_t \\ v_t \end{bmatrix},$$

即观测矩阵为

$$\mathbf{H}_t = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

由于在实际场景中，目标的控制输入往往不可知，因此通常假设其为零向量，即

2269 $\mathbf{u}_t = \mathbf{0}$ 。此时，卡尔曼滤波的模型可简化为

$$2270 \quad \begin{cases} \mathbf{x}_t = \mathbf{F}_t \mathbf{x}_{t-1} + \mathbf{w}_t \\ \mathbf{z}_t = \mathbf{H}_t \mathbf{x}_t + \mathbf{v}_t \end{cases}.$$

2271 与最小均方滤波不同，卡尔曼滤波通常对系统的状态向量和观测向量具有明确的
2272 物理含义。因此，其状态转移矩阵和观测矩阵往往是已知的，或可通过物理机理建模
2273 得到，而无需像最小均方滤波那样从数据中进行估计。卡尔曼滤波的核心在于基于已
2274 有观测对预测误差和估计误差进行统计建模，从而提升状态估计的精度与鲁棒性。总
2275 体来看，其处理流程仍遵循“先预测、后校正”的基本思想。

2276 给定 T 时刻的状态向量和观测向量 $\mathbf{x}_T \in \mathbb{R}^{M \times 1}$ 和 $\mathbf{z}_T \in \mathbb{R}^{N \times 1}$ ，与此同时，我们
2277 可以利用状态转移模型利用前一时刻状态的估计对当前状态进行预测（方便起见，状
2278 态转移矩阵和观测矩阵均去掉下标，用 $\mathbf{F}$ 和 $\mathbf{H}$ 表示）：

$$2279 \quad \hat{\mathbf{x}}_T = \mathbf{F} \hat{\mathbf{x}}_{T-1}$$

2280 再利用观测矩阵对当前状态进行预测，得到观测向量为：

$$2281 \quad \hat{\mathbf{z}}_T = \mathbf{H} \hat{\mathbf{x}}_T.$$

2282 由于观测向量受到观测噪声的影响，因此预测值与实际观测值之间存在误差，即观测
2283 向量的预测误差为：

$$2284 \quad \mathbf{e}_T = \mathbf{z}_T - \hat{\mathbf{z}}_T.$$

2285 遵循“先预测、后修正”的思想，卡尔曼滤波通过观测向量的预测误差对状态向量进
2286 行修正，得到修正后的状态向量，也即最终的估计为

$$2287 \quad \tilde{\mathbf{x}}_T = \hat{\mathbf{x}}_T + \mathbf{K}_T \mathbf{e}_T,$$

2288 其中， $\mathbf{K}_T \in \mathbb{R}^{M \times N}$ 为一个待求解的卡尔曼增益矩阵。

2289 显然，我们希望最终修正后的状态向量与实际状态向量之间的误差尽可能小，即

$$2290 \quad \min_{\mathbf{K}_T} \|\tilde{\mathbf{x}}_T - \mathbf{x}_T\|^2 = \min_{\mathbf{K}_T} \|\hat{\mathbf{x}}_T - \mathbf{x}_T + \mathbf{K}_T \mathbf{e}_T\|^2.$$

2291 该问题同样是一个最小二乘问题，其解析解为

$$2292 \quad \mathbf{K}_T = (\mathbf{x}_T - \hat{\mathbf{x}}_T) \mathbf{e}_T^T (\mathbf{e}_T \mathbf{e}_T^T)^{-1}.$$

2293 注意到预测误差可以表示为

$$2294 \quad \mathbf{e}_T = \mathbf{z}_T - \hat{\mathbf{z}}_T = \mathbf{H} \mathbf{x}_T + \mathbf{v}_T - \mathbf{H} \hat{\mathbf{x}}_T = \mathbf{H}(\mathbf{x}_T - \hat{\mathbf{x}}_T) + \mathbf{v}_T = \mathbf{H} \boldsymbol{\epsilon}_T + \mathbf{v}_T,$$

2295 其中， $\boldsymbol{\epsilon}_T = \mathbf{x}_T - \hat{\mathbf{x}}_T$ 为状态向量的预测误差。与此同时，通常可以假设状态预测误

差 ϵ_T 和观测噪声 v_T 是相互独立的, 即 $\epsilon_T \perp v_T$ 。此时, 增益矩阵可以表示为

$$\begin{aligned} \mathbf{K}_T &= \epsilon_T (\mathbf{H}\epsilon_T + v_T)^T ((\mathbf{H}\epsilon_T + v_T)(\mathbf{H}\epsilon_T + v_T)^T)^{-1} \\ &= \epsilon_T \epsilon_T^T \mathbf{H}^T (\mathbf{H}\epsilon_T \epsilon_T^T \mathbf{H}^T + v_T v_T^T)^{-1}. \end{aligned}$$

方便起见, 记状态预测误差的协方差矩阵为 $\mathbf{S}_T = \mathbb{E}[\epsilon_T \epsilon_T^T]$, 观测噪声的协方差矩阵为 $\mathbf{R} = \mathbb{E}[v_T v_T^T]$, 通常为已知的或可通过实验测量得到的。则期望的卡尔曼增益矩阵可以表示为 (方便起见, 直接记 $\mathbf{K}_T$ 为期望的卡尔曼增益矩阵 $\mathbb{E}[\mathbf{K}_T]$):

$$\mathbf{K}_T = \mathbf{S}_T \mathbf{H}^T (\mathbf{H} \mathbf{S}_T \mathbf{H}^T + \mathbf{R})^{-1}.$$

在该表达式中, 矩阵 $\mathbf{S}_T$ 是一个未知量, 需要对其进行进一步的估计。注意到

$$\epsilon_T = \mathbf{x}_T - \hat{\mathbf{x}}_T = \mathbf{F} \mathbf{x}_{T-1} + \mathbf{w}_T - \mathbf{F} \tilde{\mathbf{x}}_{T-1} = \mathbf{F} \epsilon_{T-1} + \mathbf{w}_T,$$

其中, $\epsilon_{T-1} = \mathbf{x}_{T-1} - \tilde{\mathbf{x}}_{T-1}$ 为前一时刻的状态估计误差。与此同时, 通常可以假设状态估计误差 ϵ_{T-1} 和过程噪声 $\mathbf{w}_T$ 是相互独立的, 即 $\epsilon_{T-1} \perp \mathbf{w}_T$ 。因此, 状态预测误差的协方差矩阵可以递推表示为

$$\mathbf{S}_T = \mathbb{E}[\epsilon_T \epsilon_T^T] = \mathbf{F} \mathbb{E}[\epsilon_{T-1} \epsilon_{T-1}^T] \mathbf{F}^T + \mathbb{E}[\mathbf{w}_T \mathbf{w}_T^T] = \mathbf{F} \mathbf{P}_{T-1} \mathbf{F}^T + \mathbf{Q},$$

其中, $\mathbf{P}_{T-1} = \mathbb{E}[\epsilon_{T-1} \epsilon_{T-1}^T]$ 为前一时刻的状态估计误差的协方差矩阵, 而 $\mathbf{Q} = \mathbb{E}[\mathbf{w}_T \mathbf{w}_T^T]$ 为过程噪声的协方差矩阵, 通常为已知的或可通过实验测量得到。此外, 对于状态估计误差 ϵ_T , 我们有

$$\begin{aligned} \epsilon_T &= \mathbf{x}_T - \tilde{\mathbf{x}}_T = \mathbf{x}_T - \hat{\mathbf{x}}_T - \mathbf{K}_T e_T \\ &= \epsilon_T - \mathbf{K}_T (\mathbf{H} \epsilon_T + v_T) \\ &= (\mathbf{I} - \mathbf{K}_T \mathbf{H}) \epsilon_T - \mathbf{K}_T v_T. \end{aligned}$$

因此, 协方差矩阵 $\mathbf{P}_T$ 可以表示为

$$\begin{aligned} \mathbf{P}_T &= \mathbb{E}[\epsilon_T \epsilon_T^T] = \mathbb{E}[(\mathbf{I} - \mathbf{K}_T \mathbf{H}) \epsilon_T - \mathbf{K}_T v_T] ((\mathbf{I} - \mathbf{K}_T \mathbf{H}) \epsilon_T - \mathbf{K}_T v_T)^T \\ &= (\mathbf{I} - \mathbf{K}_T \mathbf{H}) \mathbf{S}_T (\mathbf{I} - \mathbf{K}_T \mathbf{H})^T + \mathbf{K}_T \mathbf{R} \mathbf{K}_T^T \\ &= \mathbf{S}_T - \mathbf{K}_T \mathbf{H} \mathbf{S}_T - \mathbf{S}_T \mathbf{H}^T \mathbf{K}_T^T + \mathbf{K}_T (\mathbf{H} \mathbf{S}_T \mathbf{H}^T + \mathbf{R}) \mathbf{K}_T^T, \end{aligned}$$

将 $\mathbf{K}_T$ 的表达式代入到上式中最后一个 $\mathbf{K}_T$ 中, 并进行化简, 可得

$$\begin{aligned} \mathbf{P}_T &= \mathbf{S}_T - \mathbf{K}_T \mathbf{H} \mathbf{S}_T - \mathbf{S}_T \mathbf{H}^T \mathbf{K}_T^T + \mathbf{K}_T (\mathbf{H} \mathbf{S}_T \mathbf{H}^T + \mathbf{R}) (\mathbf{H} \mathbf{S}_T \mathbf{H}^T + \mathbf{R})^{-1} \mathbf{H} \mathbf{S}_T \\ &= \mathbf{S}_T - \mathbf{K}_T \mathbf{H} \mathbf{S}_T - \mathbf{S}_T \mathbf{H}^T \mathbf{K}_T^T + \mathbf{K}_T \mathbf{H} \mathbf{S}_T \\ &= \mathbf{S}_T - \mathbf{K}_T \mathbf{H} \mathbf{S}_T = (\mathbf{I} - \mathbf{K}_T \mathbf{H}) \mathbf{S}_T. \end{aligned}$$

至此, 所有未知量均已用已知量加以表达, 卡尔曼滤波的递推过程也随之得以完整刻画。综上, 其递推算法如 算法 6.1 所示。

例 6.2 利用卡尔曼滤波算法对目标轨迹进行预测, 目标轨迹如图 6.1 所示。

算法 6.1 卡尔曼滤波递推算法

输入： 状态转移矩阵 $\mathbf{F}$, 观测矩阵 $\mathbf{H}$, 过程噪声协方差 $\mathbf{Q}$, 观测噪声协方差 $\mathbf{R}$; 初始状态估计 $\tilde{\mathbf{x}}_0$, 初始估计误差协方差矩阵 $\mathbf{P}_0$

输出： 每一时刻的状态估计 $\tilde{\mathbf{x}}_T$

- 1: **初始化：** $\hat{\mathbf{x}}_1 = \mathbf{F}\tilde{\mathbf{x}}_0$, $\mathbf{S}_1 = \mathbf{F}\mathbf{P}_0\mathbf{F}^T + \mathbf{Q}$, $\mathbf{K}_1 = \mathbf{S}_1\mathbf{H}^T (\mathbf{H}\mathbf{S}_1\mathbf{H}^T + \mathbf{R})^{-1}$
- 2: **for** $T = 1, 2, \dots$ **do**
- 3: 更新当前状态估计: $\tilde{\mathbf{x}}_T = \hat{\mathbf{x}}_T + \mathbf{K}_T (z_T - \mathbf{H}\hat{\mathbf{x}}_T)$
- 4: 预测未来状态: $\hat{\mathbf{x}}_{T+1} = \mathbf{F}\tilde{\mathbf{x}}_T$
- 5: 更新状态估计误差矩阵: $\mathbf{P}_T = (\mathbf{I} - \mathbf{K}_T\mathbf{H})\mathbf{S}_T$
- 6: 更新状态预测误差矩阵: $\mathbf{S}_{T+1} = \mathbf{F}\mathbf{P}_T\mathbf{F}^T + \mathbf{Q}$
- 7: 更新卡尔曼增益: $\mathbf{K}_{T+1} = \mathbf{S}_{T+1}\mathbf{H}^T (\mathbf{H}\mathbf{S}_{T+1}\mathbf{H}^T + \mathbf{R})^{-1}$
- 8: **end for**

解 预测结果如图 6.4 所示。可以看到，由于内置了匀速直线运动的状态转移矩阵，卡尔曼滤波算法收敛速度较快，且能够较好地适应轨迹的变化。并且，如果进一步引入加速度等控制输入，则可以进一步提升预测的准确性。

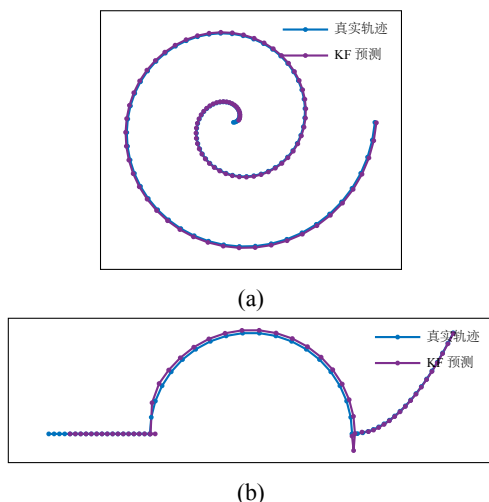


图 6.4 卡尔曼滤波预测结果

6.2 数据关联

在实际场景中，往往存在多个目标同时运动的情况。此时，观测数据中包含了多个目标的观测值，还有可能存在虚警。因此，除了需要对每个目标的轨迹进行预测之外，还需要对观测数据进行关联，从而确定每个观测值对应的是哪个目标，这一过程即为数据关联 (Data Association)。

6.2.1 联合概率数据关联

联合概率数据关联 (Joint Probabilistic Data Association, JPDA) 算法是一种经典的数据关联方法, 在介绍该算法之前, 我们首先介绍其单目标的版本, 即概率数据关联 (Probabilistic Data Association, PDA) 算法。

考虑上文所述先预测、再修正的滤波框架。对于 T 时刻的观测向量 $\mathbf{z}_T$, 假设对应的预测为 $\hat{\mathbf{x}}_T$ 。那么修正后的状态估计为

$$\tilde{\mathbf{x}}_T = \hat{\mathbf{x}}_T + \mathbf{K}_T(\mathbf{z}_T - \hat{\mathbf{z}}_T),$$

其中, $\mathbf{K}_T$ 为增益矩阵。

然而在实际场景中, 受虚警率等因素影响, 可能存在多个观测向量, 记为

$$\{\mathbf{z}_T^{(1)}, \mathbf{z}_T^{(2)}, \dots, \mathbf{z}_T^{(M)}\},$$

其中, M 为观测的数量。此时, 需确定哪一个观测值源自目标。最简单的方法是, 选择与预测值距离最近的观测值作为真正的观测值, 即

$$m^* = \arg \min_{j=1,2,\dots,M} \|\mathbf{z}_T^{(j)} - \hat{\mathbf{z}}_T\|^2,$$

然后利用该观测值进行修正:

$$\tilde{\mathbf{x}}_T = \hat{\mathbf{x}}_T + \mathbf{K}_T(\mathbf{z}_T^{(m^*)} - \hat{\mathbf{z}}_T).$$

这种方法称为最近邻 (Nearest Neighbor, NN) 方法, 其缺点在于, 忽略了其他观测值的信息, 且当虚警较多时, 容易选择错误的观测值, 从而导致估计误差较大。而 PDA 算法则通过对所有观测值进行加权平均, 从而提升估计的准确性与鲁棒性。令 β_m 为第 m 个观测值与预测值之间的关联概率, 则 PDA 算法的修正公式为

$$\tilde{\mathbf{x}}_T = \hat{\mathbf{x}}_T + \mathbf{K}_T \sum_{m=1}^M \beta_m (\mathbf{z}_T^{(m)} - \hat{\mathbf{z}}_T).$$

需要注意的是, 关联概率需要满足归一化条件, 即

$$\sum_{m=0}^M \beta_m = 1,$$

其中, β_0 为目标未被观测到的概率, 通常为一个预设值。此外, 关联概率有多种计算方法, 比如基于距离的计算方法: 距离越近, 关联概率越大; 基于统计的计算方法: 假设观测误差服从某种分布, 则根据预测值与观测值之间的误差, 计算其概率密度, 从而得到关联概率。

当存在多个目标时, 可以利用 JPDA 算法对所有目标的观测值进行联合建模, 从而实现多目标的数据关联。具体来说, 假设存在 N 个目标, 每个目标在 T 时刻的状

态预测值分别为

$$\{\hat{\mathbf{x}}_T^{(1)}, \hat{\mathbf{x}}_T^{(2)}, \dots, \hat{\mathbf{x}}_T^{(N)}\},$$

对应的观测值为

$$\{\mathbf{z}_T^{(1)}, \mathbf{z}_T^{(2)}, \dots, \mathbf{z}_T^{(M)}\}.$$

那么对于第 n 个目标，其修正后的状态估计为

$$\tilde{\mathbf{x}}_T^{(n)} = \hat{\mathbf{x}}_T^{(n)} + \mathbf{K}_T^{(n)} \sum_{m=1}^M \beta_{mn} (\mathbf{z}_T^{(m)} - \hat{\mathbf{z}}_T^{(n)}),$$

其中， $\mathbf{K}_T^{(n)}$ 为第 n 个目标的增益矩阵， β_{mn} 为第 m 个观测值与第 n 个目标之间的关联概率。需要注意的是，关联概率需要满足归一化条件，即

$$\sum_{m=0}^M \beta_{mn} = 1,$$

其中， β_{0n} 为第 n 个目标未被观测到的概率。此外，关联概率还需要满足

$$\sum_{n=1}^N \beta_{mn} \leq 1, \quad m = 1, 2, \dots, M.$$

该约束表明，对于一个观测 $\mathbf{z}_T^{(m)}$ ，其分配到各目标的概率之和不能超过 1，也就是说一个观测值最多只能来源于一个目标。对于 PDA 和 JPDA 算法，增益矩阵的具体迭代过程较为复杂，此处不再展开，读者可参考原始文献 [1, 2]。

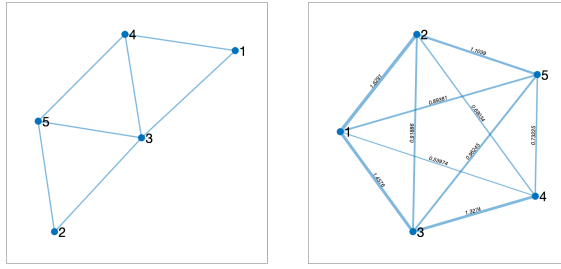
为进一步提升跟踪的精度与鲁棒性，多假设跟踪（Multiple Hypothesis Tracking, MHT）算法[3]在此基础上引入了“多假设”的思想。MHT 在每一个时间步都会为新的观测生成多种可能的关联假设，并对这些假设进行评分与筛选，逐步保留最有可能的轨迹，从而有效解决目标交叉、遮挡等复杂场景下的跟踪问题。

6.2.2 图匹配方法

图匹配（Graph Matching）方法是一类基于图论的多目标跟踪技术，其核心思想是将多目标跟踪问题转化为图匹配问题，并借助图论中的理论与算法加以求解。在展开具体方法之前，有必要先对图（graph）的基本概念进行介绍。

定义 6.1 一个大小为 n 的无向图 $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ 由一个离散的节点集合 $\mathcal{V} = \{\mathcal{V}_i\}_{i=1}^n$ 和一个无向边集合 $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ 组成，满足 $\mathcal{E}_{ii'} = \mathcal{E}_{i'i}$ 。

通常，无向图的边集合 $\mathcal{E}$ 可以用一个对称的矩阵 $\mathbf{E} \in \mathbb{R}^{n \times n}$ 表示，其中的元素表示节点之间的连接关系或权重。例如，图 6.5 所示的两个图，其边的集合对应的矩阵



(a) 无权重无向图

(b) 有权重无向图

图 6.5 无向图示例

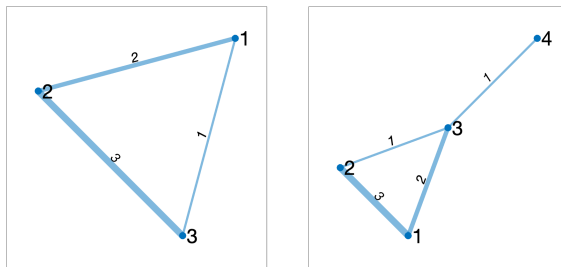
分别为

$$\mathbf{E}_1 = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}, \quad \mathbf{E}_2 = \begin{bmatrix} 0.00 & 1.63 & 1.46 & 0.54 & 0.89 \\ 1.63 & 0.00 & 0.92 & 0.68 & 1.10 \\ 1.46 & 0.92 & 0.00 & 1.33 & 0.96 \\ 0.54 & 0.68 & 1.33 & 0.00 & 0.73 \\ 0.89 & 1.10 & 0.96 & 0.73 & 0.00 \end{bmatrix}.$$

在许多实际应用中，图 $\mathcal{G}$ 往往伴随有以向量形式表示的节点与边的属性。对于这样的属性图 (Attribute Graph)，我们记 $\mathbf{v}_i \in \mathbb{R}^{d_v}$ 为节点 $\mathcal{V}_i$ 的属性向量， $\mathbf{e}_{ii'} \in \mathbb{R}^{d_e}$ 为边 $\mathcal{E}_{ii'}$ 的属性向量。与此同时，其邻接矩阵 $\mathbf{E}$ 可以通过某些特定的函数计算得到，例如

$$\mathbf{E}_{ii'} = \phi(\mathbf{v}_i, \mathbf{v}_{i'}) \quad \text{或} \quad \mathbf{E}_{ii'} = \phi(\mathbf{e}_{ii'}).$$

给定两个图 $\mathcal{G}^1 = \{\mathcal{V}^1, \mathcal{E}^1\}$ 与 $\mathcal{G}^2 = \{\mathcal{V}^2, \mathcal{E}^2\}$ ，其节点数分别为 m 与 n (不妨假设 $m \leq n$)，图匹配 (Graph Matching, GM) 问题旨在寻找一种最优的节点对应关系。



(a)

(b)

图 6.6 图匹配示例

如图 6.6 所示的两个图，不难发现它们节点之间的对应关系如下：

$$\begin{bmatrix} 1 & \rightarrow & 3 \\ 2 & \rightarrow & 1 \\ 3 & \rightarrow & 2 \\ \emptyset & \rightarrow & 4 \end{bmatrix},$$

其中 $\emptyset \rightarrow 4$ ，表明图 6.6a 中没有节点与图 6.6b 中的节点 4 对应。为了形式化地描述图匹配问题，我们引入匹配矩阵的概念。具体而言，我们希望构造一个匹配矩阵

$$\mathbf{P} \in \{0, 1\}^{m \times n},$$

其第 i 行第 j 列元素 $p_{ij} = 1$ ，则表示 $\mathcal{G}^1$ 中的节点 $\mathcal{V}_i^1$ 与 $\mathcal{G}^2$ 中的节点 $\mathcal{V}_j^2$ 相匹配；反之 $p_{ij} = 0$ 。因此，对于图 6.6 所示的两个图，其匹配矩阵为

$$\mathbf{P} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

此外，为保证匹配的合理性，矩阵 $\mathbf{P}$ 通常需要满足一定的约束条件，记 $\mathbf{P}$ 必然属于集合 $\mathcal{P}$ ：

$$\mathcal{P} := \{\mathbf{P} \in \{0, 1\}^{m \times n} | \mathbf{P}\mathbf{1}_n = \mathbf{1}_m, \mathbf{P}^T\mathbf{1}_m \leq \mathbf{1}_n\}$$

其中 $\mathbf{1}_k$ 表示维度为 k 的全 1 向量。前一个约束确保 $\mathcal{G}^1$ 中的每个节点都唯一匹配到 $\mathcal{G}^2$ 中的一个节点，后一个约束则避免多个节点同时映射到 $\mathcal{G}^2$ 的同一节点。

考虑两个图节点之间的相似性，我们可以定义一个相似性矩阵 $\mathbf{S} \in \mathbb{R}^{m \times n}$ ，其元素 s_{ij} 衡量 $\mathcal{G}^1$ 中节点 $\mathcal{V}_i^1$ 与 $\mathcal{G}^2$ 中节点 $\mathcal{V}_j^2$ 之间的相似度。通常，节点属性越相似，其对应的相似度值也应越大。基于此，我们可以将图匹配问题形式化为以下优化问题：

$$\begin{cases} \max_{\mathbf{P}} \sum_{i=1}^m \sum_{j=1}^n p_{ij} s_{ij} \\ \text{s.t. } \mathbf{P} \in \mathcal{P} \end{cases}.$$

该问题旨在通过选择合适的匹配矩阵 $\mathbf{P}$ ，最大化匹配节点之间的总相似度。可以利用一些优化算法（如匈牙利算法、线性规划等）来求解该问题，这类算法比较成熟，可直接调用相关库实现。下面，我们介绍一种快速的近似算法——重加权随机游走（Reweighted Random Walks, RRWM）算法。注意到，目标函数可以表示为

$$\sum_{i=1}^m \sum_{j=1}^n p_{ij} s_{ij} = \text{vec}(\mathbf{P})^T \text{vec}(\mathbf{S}),$$

其中, $\text{vec}(\cdot)$ 表示矩阵的向量化操作。又因为 $\mathbf{P}$ 是一个二值矩阵, 因此目标函数可以进一步表示为如下的二次形式:

$$\text{vec}(\mathbf{P})^T \text{diag}(\text{vec}(\mathbf{S})) \text{vec}(\mathbf{P}) = \text{vec}(\mathbf{P})^T \mathbf{W} \text{vec}(\mathbf{P}),$$

其中, $\mathbf{W} = \text{diag}(\text{vec}(\mathbf{S}))$ 为一个对角矩阵。注意到, 在不考虑约束的情况下, 目标函数取得最大值时, $\text{vec}(\mathbf{P})$ 为 $\mathbf{W}$ 的最大特征值对应的特征向量。因此, RRWM 算法的核心思想是, 通过迭代计算特征向量, 并结合约束条件, 对特征向量进行重加权与归一化, 从而逐步逼近最优解。具体而言, RRWM 算法的迭代过程如下:

1. 初始化: 设定初始矩阵 $\mathbf{P}^{(0)} = \frac{1}{mn} \mathbf{1}_{mn}$, 迭代次数 $t = 0$ 。
2. 迭代计算: 计算 $\text{vec}(\mathbf{P}^{(t+1)}) = \mathbf{W} \text{vec}(\mathbf{P}^{(t)})$ 。
3. 重加权与归一化: 对 $\mathbf{P}^{(t+1)}$ 的每一行和列交替进行 Softmax 操作多次, 直到收敛为止:

$$p_{ij}^{(t+1)} = \frac{\exp(p_{ij}^{(t+1)})}{\sum_{k=1}^n \exp(p_{ik}^{(t+1)})}, \quad p_{ij}^{(t+1)} = \frac{\exp(p_{ij}^{(t+1)})}{\sum_{k=1}^m \exp(p_{kj}^{(t+1)})}$$

4. 如果满足收敛条件或达到最大迭代次数, 则停止迭代; 否则, 令 $t = t + 1$, 返回步骤 2。

迭代结束后, 得到的匹配矩阵 $\mathbf{P}^{(t)}$ 可以重新构造匹配矩阵 $\mathbf{P}$, 并通过阈值化操作将其转换为二值矩阵, 从而得到最终的匹配结果。

上文所构建的图匹配问题只考虑了节点之间的相似性, 因此也被称作线性分配问题 (Linear Assignment Problem, LAP)。然而, 边之间的相似性同样重要。为此, 我们可以将图匹配问题扩展为二次分配问题 (Quadratic Assignment Problem, QAP), 对应的图匹配问题可以表述为如下形式:

$$\begin{cases} \max_{\mathbf{P}} & \sum_{ij} p_{ij} k_{iijj} + \sum_{ii'jj'} k_{ii'jj'} p_{ij} p_{i'j'} \\ \text{s.t.} & \mathbf{P} \in \mathcal{P}, \end{cases}$$

其中, k_{iijj} 表示节点 $\mathcal{V}_i^1$ 与 $\mathcal{V}_j^2$ 的相似度, $k_{ii'jj'}$ 表示边 $\mathcal{E}_{ii'}^1$ 与 $\mathcal{E}_{jj'}^2$ 的相似度。对于节点而言, 只要 $p_{ij} = 1$, 对应的节点相似度就会被计入目标函数中。而对于边而言, 只有当两个图中的边所连接的节点均被匹配时, 其相似度才会被计入目标函数中, 即 $p_{ij} = p_{i'j'} = 1$ 时, 才会计入边相似度。

方便起见, 我们可以构建矩阵 $\mathbf{K} \in \mathbb{R}^{mn \times mn}$, 该矩阵也被称为图 $\mathcal{G}^1$ 与 $\mathcal{G}^2$ 的亲矩阵 (Affinity Matrix)。对于亲和力 $\mathbf{K}$ 中的元素, 有如下定义:

$$(\mathbf{K})_{m(i-1)+j, m(i'-1)+j'} = k_{ii'jj'}.$$

基于此, 我们可以将目标函数重新表示为

$$\text{vec}(\mathbf{P})^T \mathbf{K} \text{vec}(\mathbf{P}).$$

该问题仍然可以通过 RRWM 算法进行求解，只需将 $\mathbf{W}$ 替换为 $\mathbf{K}$ 即可。

在数据关联问题中，我们可以将预测的目标状态与实际观测值分别构建为两个图，然后通过图匹配算法确定它们之间的对应关系，从而实现数据关联。并且，对于图中的节点，我们可以将其位置、速度等信息作为节点属性，从而计算节点之间的相似度。此外，边的属性可以表示目标之间的相对位置、相对速度等信息，从而计算边之间的相似度。通过这种方式，图匹配方法能够有效地利用目标之间的空间与运动关系，从而提升数据关联的准确性与鲁棒性。

在前文构建的图匹配问题中，仅考虑了节点之间的相似性，因此该问题也被称为线性分配问题 (Linear Assignment Problem, LAP)。然而，在实际应用中，边的相似性同样具有重要作用。为此，可以将图匹配问题进一步扩展为二次分配问题 (Quadratic Assignment Problem, QAP)，其数学形式为

$$\begin{cases} \max_{\mathbf{P}} & \sum_{ij} p_{ij} k_{iijj} + \sum_{ii'jj'} k_{ii'jj'} p_{ij} p_{i'j'} \\ \text{s.t.} & \mathbf{P} \in \mathcal{P} \end{cases},$$

其中， k_{iijj} 表示节点 $\mathcal{V}_i^1$ 与 $\mathcal{V}_j^2$ 的相似度； $k_{ii'jj'}$ 表示边 $\mathcal{E}_{ii'}^1$ 与 $\mathcal{E}_{jj'}^2$ 的相似度。具体来说，对于节点而言，当 $p_{ij} = 1$ 时，对应节点的相似度将被计入目标函数；而对于边而言，仅当两条边的端点均被正确匹配时，即 $p_{ij} = p_{i'j'} = 1$ ，其边相似度才会被计入目标函数。

为便于表述，可以构造一个亲和矩阵 (Affinity Matrix) $\mathbf{K} \in \mathbb{R}^{mn \times mn}$ 。其元素定义如下：

$$(\mathbf{K})_{m(i-1)+j, m(i'-1)+j'} = k_{ii'jj'}.$$

在此定义下，目标函数可紧凑地写作

$$\text{vec}(\mathbf{P})^T \mathbf{K} \text{vec}(\mathbf{P}).$$

该问题同样可以通过 RRWM 算法进行求解，只需将其中的权重矩阵 $\mathbf{W}$ 替换为亲和矩阵 $\mathbf{K}$ 即可。

在数据关联问题中，我们可以将预测的目标状态与实际观测值分别建模为两幅图，并通过图匹配算法确定它们之间的对应关系，从而实现数据关联。具体而言，节点属性可包含目标的位置、速度等信息，用于计算节点间的相似度；边的属性则可刻画目标之间的相对位置或相对速度，用于计算边之间的相似度。通过这种方式，图匹配方法能够综合利用目标之间的空间与运动关系，从而显著提升数据关联的准确性与鲁棒性。

参考文献

- [1] FORTMANN Thomas-E., BAR-SHALOM Yaakov, SCHEFFE Molly. Multi-target tracking using joint probabilistic data association[C]//1980 19th IEEE Conference on Decision and Control including the Symposium on Adaptive Processes. 1980: 807-812.
- [2] BAR-SHALOM Yaakov, TSE Edison. Tracking in a cluttered environment with probabilistic data association[J]. Automatica, 1975, 11(5): 451-460.
- [3] BLACKMAN S.S. Multiple hypothesis tracking for multiple target tracking[J]. IEEE Aerospace and Electronic Systems Magazine, 2004, 19(1): 5-18.